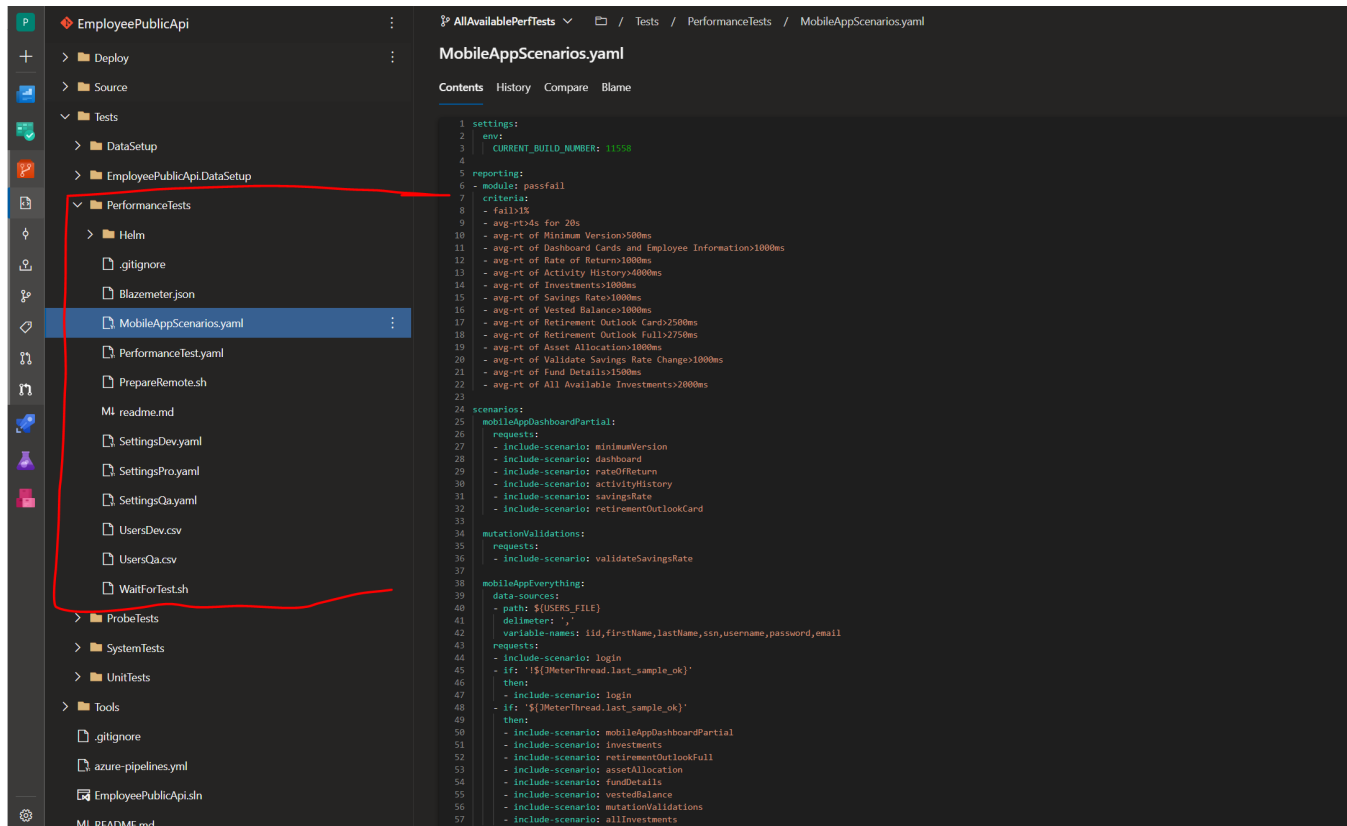


Test Organization

One of the best features of Taurus is the ability to separate all of your configuration files. Instead of one big Jmeter XML file that is virtually impossible to decipher in source control, you can split up your files in Taurus in such a way that is easily readable and maintainable.

See this sample repo for the Employee Public API to see how the tests are organized.

https://tfs.ascensus.com/tfs/PSG/Portfolio/_git/EmployeePublicApi?path=%2FTests%2FPerformanceTests

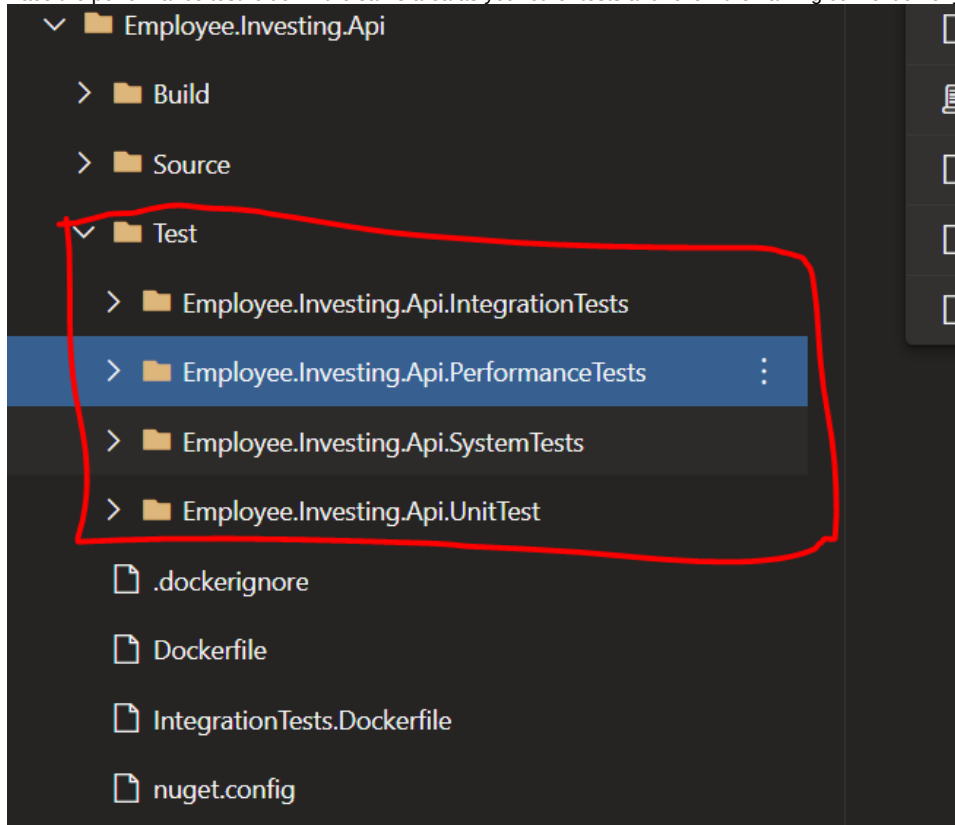


The screenshot displays a Git repository interface. On the left, the file explorer shows the directory structure: `EmployeePublicApi` (root) contains `Deploy`, `Source`, `Tests`, `DataSetup`, `EmployeePublicApi.DataSetup`, `PerformanceTests`, `ProbeTests`, `SystemTests`, `UnitTests`, and `Tools`. The `PerformanceTests` directory is expanded, showing files: `.gitignore`, `Blazemeter.json`, `MobileAppScenarios.yaml` (selected), `PerformanceTest.yaml`, `PrepareRemote.sh`, `MI_readme.md`, `SettingsDev.yaml`, `SettingsPro.yaml`, `SettingsQa.yaml`, `UsersDev.csv`, `UsersQa.csv`, and `WaitForTest.sh`. The right pane shows the content of `MobileAppScenarios.yaml`, which is a YAML file defining test settings, reporting criteria, and scenarios.

```
1 settings:
2   env:
3     CURRENT_BUILD_NUMBER: 11558
4
5 reporting:
6   - module: passfail
7   criteria:
8     - fail>1%
9     - avg-rt>4s for 20s
10    - avg-rt of Minimum Version>500ms
11    - avg-rt of Dashboard Cards and Employee Information>1000ms
12    - avg-rt of Rate of Return>1000ms
13    - avg-rt of Activity History>4000ms
14    - avg-rt of Investments>1000ms
15    - avg-rt of Savings Rate>1000ms
16    - avg-rt of Vested Balance>1000ms
17    - avg-rt of Retirement Outlook Cards>2500ms
18    - avg-rt of Retirement Outlook Full>2750ms
19    - avg-rt of Asset Allocation>1800ms
20    - avg-rt of Validate Savings Rate Change>1000ms
21    - avg-rt of Fund Details>1500ms
22    - avg-rt of All Available Investments>2000ms
23
24 scenarios:
25   mobileAppDashboardPartial:
26     requests:
27       - include-scenario: minimumVersion
28       - include-scenario: dashboard
29       - include-scenario: rateOfReturn
30       - include-scenario: activityHistory
31       - include-scenario: savingsRate
32       - include-scenario: retirementOutlookCard
33
34   mutationValidations:
35     requests:
36       - include-scenario: validateSavingsRate
37
38   mobileAppEverything:
39     data-sources:
40       - path: ${USERS_FILE}
41       delimiter: ','
42       variable-names: iid,firstName,lastName,ssn,username,password,email
43     requests:
44       - include-scenario: login
45       - if: '${JMeterThread.last_sample_ok}'
46         then:
47           - include-scenario: login
48       - if: '${JMeterThread.last_sample_ok}'
49         then:
50           - include-scenario: mobileAppDashboardPartial
51           - include-scenario: investments
52           - include-scenario: retirementOutlookFull
53           - include-scenario: assetAllocation
54           - include-scenario: fundDetails
55           - include-scenario: vestedBalance
56           - include-scenario: mutationValidations
57           - include-scenario: allInvestments
```

There are no strict rules but here are some guidelines

- Place the performance test folder in the same area as your other tests and follow the naming convention of your git repo



- Put all of your environmental settings into different files. In the example above, you can see there is a SettingsDev.yaml, SettingsQa.yaml, and SettingsPro.yaml. This allows you to easily switch environments by simply swapping whatever settings file you want to run in the bzt command

Running in QA

```
bzt PerformanceTest.yaml MobileAppScenarios.yaml SettingsQa.yaml
```

Running in Production

```
bzt PerformanceTest.yaml MobileAppScenarios.yaml SettingsPro.yaml
```

- Have one or more files dedicated to your scenarios only and no other settings. In the above example, all of the scenarios are in one file called MobileAppScenarios.yaml. The amount of scenarios is not overwhelming, thus it is easily maintained in one file.

- Have one file specifically for test execution settings only. In this case, the file is called PerformanceTest.yaml.

PerformanceTest.yaml

[Contents](#) [History](#) [Compare](#) [Blame](#)

```
1 settings:
2   artifacts-dir: TestArtifacts
3
4 execution:
5   - concurrency: 1
6     ramp-up: 30s
7     hold-for: 600s
8     scenario: mobileAppEverything
9
```

- Utilize the include-scenario directive to promote reusability. The biggest downside with Jmeter is you cannot reuse anything. Everything has to be duplicated everywhere. The beauty of Taurus is that you can nest scenarios into other scenarios, which makes this clean and tidy.

```
scenarios:
  mobileAppDashboardPartial:
    requests:
      - include-scenario: minimumVersion
      - include-scenario: dashboard
      - include-scenario: rateOfReturn
      - include-scenario: activityHistory
      - include-scenario: savingsRate
      - include-scenario: retirementOutlookCard

  mutationValidations:
    requests:
      - include-scenario: validateSavingsRate

  mobileAppEverything:
    data-sources:
      - path: ${USERS_FILE}
        delimiter: ','
        variable-names: iid,firstName,lastName,ssn,username,password,email
    requests:
      - include-scenario: login
      - if: '`${JMeterThread.last_sample_ok}`'
        then:
          - include-scenario: login
      - if: '`${JMeterThread.last_sample_ok}`'
        then:
          - include-scenario: mobileAppDashboardPartial
          - include-scenario: investments
          - include-scenario: retirementOutlookFull
          - include-scenario: assetAllocation
          - include-scenario: fundDetails
          - include-scenario: vestedBalance
          - include-scenario: mutationValidations
          - include-scenario: allInvestments
```